FYP Development Roadmap

Referral Preparation and Follow through Platform

A practical implementation guide for a three member team using AI coding agents. Build one complete referral journey first, then add document intelligence and medication flags without weakening patient linking, source permissions, or clinician approval.

1 Architecture and system boundaries

The system has a central modular application and three independent source services. Clinic, laboratory, and specialist services each own a database and make their own access decisions. They may run on one machine; independence is a service and ownership boundary, not a requirement for separate hardware.

Layer	Responsibility
Presentation	One React application with role-specific screens. Shows source evidence, uncertainty, review actions, and processing status.
API and access	FastAPI endpoints validate requests and enforce institution, role, patient, and referral permissions. Keycloak authenticates users.
Application workflow	Referral lifecycle, verified patient links, evidence selection, review requests, medication flags, versioning, approval, delivery, and follow-up.
Integration	Source-specific adapters convert local records to the agreed FHIR subset; the central importer validates and normalizes them.
Processing	A job queue and worker execute OCR, extraction, brief generation, and retryable integration jobs.
Persistence	Three source databases plus central workflow storage. Protected file storage holds PDFs and images; Redis is not the clinical source of truth.
Infrastructure	Containers, configuration, secure transport, audit logs, monitoring, backups, migrations, and repeatable seed data.

Security, audit, provenance, and versioning apply across layers. The central application calls source APIs; it never queries their databases directly. A frontend role switch is not an authorization control.

Scope follows the updated FYP SRS including FR-61 to FR-72 medication flags. Technology choices and endpoint names below are implementation proposals. Clinical rules, specialty, mapping coverage, and model quality thresholds still require the decisions described in Stage 0.

2 Data flow and FHIR integration

Where adapters live

Place a small adapter in each source service. Local SQL records enter the adapter; authorized FHIR R4 resources leave its API. The central importer converts those resources to the evidence model. The specialist adapter also translates approved referral packages and responses. Share validation utilities, but keep source-specific mappings explicit.

A complete referral journey

- Clinic user selects a local patient. The central service creates a draft and retrieves only records whose patient links and permissions are verified.
- Each source checks authorization, reads its own database, maps data through its adapter, and returns a result with source identity, record version, and clinical timestamps.
- The central importer validates resources and preserves original values. Missing, denied, unavailable, partial, empty, and invalid sources remain distinguishable.
- Uploaded documents enter protected storage. The worker extracts proposed fields; a clinician reviews them and confirms medication use and mappings.
- The deterministic evaluator applies approved medication rules and completeness checks. The language model drafts a brief from permitted evidence; flags remain separately structured.
- The clinician approves an immutable package version. The outbox sends that version to the specialist. Receipt, human acknowledgment, acceptance, flag reviews, response, and follow-up are recorded separately.

Limited FHIR R4 exchange contract

Project information	Resource
Patient and institution identity	Patient and Organization
Encounter and documented diagnosis	Encounter and Condition
Lab value and grouped report	Observation and DiagnosticReport
Prescription and reported medication use	MedicationRequest and MedicationStatement
Document metadata and evidence origin	DocumentReference and Provenance
Referral and fulfillment work	ServiceRequest and Task
Collection of exchanged resources	Bundle

Pin FHIR R4 4.0.1 and the supported fields in the repository. Validate representative resources with the HL7 validator. Pydantic shape checks alone do not establish FHIR conformance. Keep app statuses mapped explicitly to FHIR statuses.

Use a versioned application envelope for approved text, medication flags, rule versions, and acknowledgments alongside the FHIR Bundle. Label that envelope as a project API contract, not a standard FHIR resource. A full FHIR server is optional; arbitrary extra fields must not be inserted into standard resources.

3 Stack and supporting resources

Area	Proposed choice	Implementation use
Web application	React, TypeScript, Vite, MUI	Shared UI components and role-specific screens
Forms and data	React Hook Form, TanStack Query	Validation, requests, job polling, errors
Backend	FastAPI, Pydantic, HTTPX	APIs, shared schemas, authenticated service calls
Persistence	PostgreSQL, SQLAlchemy, Alembic	Four logical app databases and owned migrations
Identity	Keycloak	OIDC login; source and central permission enforcement
Jobs	Celery and Redis	Asynchronous processing, bounded retry, status
Files	Protected mounted storage initially	Original documents and extraction evidence; replaceable storage interface
OCR	One benchmarked OCR/HTR integration	Azure Read candidate or local TrOCR with required page segmentation
Language model	One model behind a provider interface	Structured extraction and source-referenced draft briefs
Rules	Python plus versioned JSON	Checklist and medication evaluation without model training
Development	Docker Compose and GitHub Actions	Local environments, builds, checks, release images
Verification	pytest and Playwright	Focused backend checks and a few browser smoke journeys

External APIs versus APIs built by the team

Build the central, clinic, lab, specialist, document, and job APIs yourselves. Keycloak provides identity endpoints. Only hosted OCR and language-model inference require external model services; local inference can replace either. Keep provider credentials on the backend.

RxNorm and RxClass can assist with medication identifiers and classes. Start with a small locally stored, reviewed mapping dictionary and keep ambiguous names unresolved. These services do not supply your clinical referral rules. Obtain clinician-reviewed references and approval for each supported rule.

Data and operating setup

Use one selected base dataset, preferably Synthea for the initial simulation unless real de-identified data is required. Partition connected patient histories across sources; generate only missing workflow records and label them synthetic. Keep a separate expected-link map for tests. Reserve representative labeled documents for AI evaluation.

Provide .env.example, pinned dependency files, health checks, named volumes, and a reset-and-seed command. Keycloak needs its own identity storage in addition to the four application databases. Do not commit patient data, credentials, large model weights, or generated uploads.

4 Three member development ownership

Use Member 1, Member 2, and Member 3 as stable ownership labels until names are assigned. Each member owns their feature backend, its screens, focused checks, and setup notes. Member 1 supplies reusable UI components so other members can implement screens without waiting for a separate frontend developer.

Member 1 Referral workflow and user experience

Owns central referral state, drafts/search, clarification lifecycle, brief editor, approvals and versions, specialist inbox screens, acknowledgment screens, response review, follow-up, shared layout, and central audit interface. Implements the database tables and API handlers for these modules.

Resources: React/MUI, FastAPI, central PostgreSQL/Alembic, shared contracts, Playwright, specialty checklist definitions. Primary areas: web/features/referrals, review, specialist, followup; central/referrals, approvals, reviews, followup, audit.

First output: shared application shell and a saved referral draft. Final output: a complete approved referral can be submitted, answered, reviewed, and closed with the prior versions intact. SRS focus: FR-11–15, FR-39–40, FR-47–54, shared FR-59–60.

Member 2 Federation and platform foundation

Owns three source services and their databases, seed/import scripts, source adapters, FHIR validation, central record retrieval, verified identity links, Keycloak configuration, permission conventions, source-level permissions, delivery adapters, source audit, Compose and initial CI. Builds identity/linking and basic source-record screens.

Resources: FastAPI/HTTPX, PostgreSQL/Alembic, FHIR R4 documentation and validator, base dataset, Keycloak, Docker. Primary areas: services/clinic, lab, specialist; central/integration, identity; web/features/identity; infra and scripts/seed.

First output: all services start with one linked synthetic test case and permitted/denied API examples. Final output: retrieval and return communication work through independent authorization boundaries. SRS focus: FR-01–10, FR-16–21, retrieval/delivery parts of FR-48–57.

Member 3 Document intelligence and medication rules

Owns uploads, storage abstraction, OCR/HTR, structured extraction, evidence correction screen, medication normalization and current-use screen, deterministic checklist/discrepancy detection, medication rule evaluator, flag evidence screen, brief generation, worker configuration, and AI evaluation.

Resources: OCR/LLM SDKs, Celery/Redis, Pydantic, document samples/transcriptions, reviewed terminology dictionary, clinician-approved rule files, pytest. Primary areas: central/documents, evidence, medications, rules, generation; worker; web/features/documents, medications; rules and evaluation.

First output: a document can be uploaded, manually transcribed, and verified, with an early OCR benchmark. Final output: reviewed evidence produces a cited draft and traceable medication flags. SRS focus: FR-22–38, FR-41–46, FR-58, FR-61–68 and FR-70–72; flag acknowledgment shared with Member 1.

5 Handoffs and shared contracts

Boundary	Owner and handoff
Specialist workflow	Member 1 owns screens and central state. Member 2 owns source storage, approval validation, receive/respond APIs, and transport.
Medication flags	Member 3 produces deterministic flags and reevaluation. Member 1 snapshots them into approved packages and stores version-specific acknowledgment. Member 2 transports them intact.
Missing information	Member 3 produces findings. Member 1 assigns, clarifies, resolves, and applies agreed submission gates.
Evidence and provenance	Member 2 imports source evidence; Member 3 imports document evidence. Both return the same EvidenceItem contract.
Authorization	Member 2 defines identity/permission conventions. Every member enforces them in owned handlers, downloads, and background jobs.
Shared central migrations	Each author creates migrations for owned tables. Member 1 coordinates central migration ordering; Member 2 owns source migrations.
Integration and releases	Member 2 maintains release scripts. All three review changes and demonstrate their module; no single member is the sole tester.

Agree these contracts before parallel coding

- PatientRef: institution_id plus local_patient_id. PatientLink: status, verifier, basis, timestamp, invalidation history. Never match solely on a name or local ID.
- EvidenceItem: evidence_id, patient reference, source institution/record/version, clinical time, retrieval/upload time, original value/text/unit, verification status, and document page/region where applicable.
- RetrievalResult: source, status (successful/empty/partial/denied/unavailable/invalid), returned evidence, error details suitable for users, and retryability.
- MedicationEntry: original name, ingredient/class, mapping source/version/status, current-use state/verifier, dates, dosage, and evidence references.
- MedicationFlag: flag ID/version, rule ID/version, medication/evidence references, specialty/procedure context, evaluation result, message, and review history.
- ReferralPackage: referral ID, immutable version, brief, selected evidence, structured flags, limitations, approver, approval time, and delivery ID. ReviewRequest and Job have separate status models.

Publish request/response examples in contracts/ and generate API documentation from the implementation. Provide deterministic stub responses immediately. Contract changes require an issue/PR and approval from the consuming member before implementation; update examples and checks in the same merge.

6 AI assisted coding and Git workflow

Repository and agent boundaries

Use one monorepo: web/, services/central/, services/clinic/, services/lab/, services/specialist/, worker/, contracts/, rules/, tests/, infra/, scripts/, and docs/. Keep feature modules inside the central application rather than creating a microservice for every function.

Add root AGENTS.md with the architecture, commands, module ownership, source access rules, and approval invariants. Add local instructions only where a module differs. Give agents the current SRS, relevant contracts, and one bounded issue. Multiple agents may work in separate branches/worktrees when files do not overlap; a human owns each result.

Standard task brief for an agent

State: goal; SRS IDs; allowed files; current input/output examples; dependencies; required authorization; failure behavior; one or two acceptance checks; and commands to run. Ask the agent to report changed files, checks run, and remaining limitations. Require coordination before schema, package, or authentication changes.

Example task: implement extraction correction in the document module. Preserve prior values, record the editor, keep clinical verification separate, enforce referral access, and add one permitted and one denied correction check. Do not change the shared evidence contract.

Daily branch and merge routine

- Create a short feature branch from current main, such as feat/m3-document-review. Commit after a coherent working change and push at the end of each work session; a push backs up work but does not deploy it.
- Open a small PR as soon as a usable slice exists. Run lint/type checks, build, affected backend checks, and contract checks. Keep main protected from direct pushes.
- One teammate reviews the contract, permissions, and acceptance evidence. Merge when the feature works; incomplete features remain hidden behind a disabled flag.
- Pull main before the next slice. Resolve migration conflicts explicitly. Review all agent edits and do not allow agents to approve their own PRs or change shared contracts silently.
- At each stage checkpoint, run the combined smoke scenario, tag the passing main commit, and deploy that exact commit when the deployment checkpoint calls for it. Avoid long-lived member branches.

What counts as done

The real endpoint and screen are connected, source ownership is respected, failure states are visible, focused checks pass, migrations work, and another member can run it from the documented setup. Mock APIs and fake model outputs must be clearly identified and removed from the demonstrated path by the corresponding stage gate.

7 Stage 0 Scope contracts and early feasibility

Entry: the three members have read the updated SRS. Exit: the first implementable scope and contracts are agreed. This stage is shared; it is not three independent design exercises.

Build and decide

- Choose one specialty and a small document set. Record supported languages, upload/page limits, medication classes, procedure contexts, and what missing fields block submission versus allow a documented clinician override.
- Identify the clinical rule/checklist reviewer. Start rule files as inactive drafts; record evidence/reference, reviewer, approval status, and version. No AI-created rule is automatically activated.
- Choose a base dataset and create one connected synthetic patient history. Assign different local IDs to all sources; retain the expected mapping only for controlled seed/test use.
- Agree API examples, status transitions, permissions, FHIR resource subset, retention ownership, and the central/source database boundary. Record decisions in docs/architecture.md.
- Run a short OCR experiment on representative document types and a summary experiment on known evidence. Include handwriting segmentation cost when considering TrOCR. Record the usable scope and failure examples before committing to a model.
- Create Git issues with owner, dependencies, SRS IDs, and acceptance behavior. Pin package/model versions after a compatibility check; do not invent delivery dates until the initial slice is measured.

Parallel responsibilities

Member 1 drafts screen flow, statuses, and approval gates. Member 2 drafts identity, source APIs, FHIR mapping, and seed structure. Member 3 benchmarks OCR/LLM and defines evidence, mapping, and rule schemas. Review contracts together once, then allow implementation against stubs.

Testing checkpoint T0

Manually trace one patient from clinic to specialist using sample JSON. Check that identifiers, evidence, medication use, and flag versions survive each handoff. Run one OCR sample and one generated brief; inspect against their expected text/facts. These checks establish feasibility, not model accuracy claims.

Git and deployment checkpoint G0

Merge docs, contracts, sample fixtures, and repository instructions through reviewed PRs. Tag the agreed starting contract as v0.0-contracts. No hosted deployment is needed yet. Keep fixture licensing and generated-versus-original data labels alongside the seed instructions.

Dependencies and stopping points

A clinical decision still awaiting review does not block the app shell, source simulation, or manual referral work. It does block activating the corresponding rule. If OCR quality is weak, bound the supported document types and retain the manual path while evaluating a replacement integration.

8 Stage 1 Running foundation and access control

Entry: shared contracts and a seed patient exist. Exit: all members can start the same environment, sign in, and reach authorized services.

Member 2 Platform and identities

Create Compose services for the web app, central API, three source APIs, worker, Redis, PostgreSQL, and Keycloak. Use separate database credentials per source and central store; provision identity storage separately. Add health/readiness endpoints and repeatable migrations. Seed institution-scoped accounts for staff, clinician, specialist, lab, and administrator roles.

Implement browser OIDC authorization-code flow with PKCE, token validation, and service audiences. Service calls carry verified actor/institution and referral context; do not trust arbitrary identity headers from the browser. Enforce local record permissions at each source. Define how disabling an account or revoking access affects active tokens and central copies; test that strategy rather than relying solely on JWT expiry.

Member 1 Application foundation

Create navigation, shared forms/tables, loading and access-denied screens. Define central Institution/User references, Referral draft, ReviewRequest, AuditEvent, and version fields. Implement an audit recording interface with actor, institution, action, target, timestamp, and outcome. Keep administrator abilities separate from clinical approval.

Member 3 Files and jobs foundation

Create a storage interface, document metadata table, upload validation, and authenticated download endpoint. Store generated object names outside the public web root; check declared and actual file type, size, and allowed formats. Add Job records and worker status updates. The first job may return a labeled fixture result; real inference comes in Stage 4.

Testing checkpoint T1

Run Compose from a fresh checkout and migrate/seed once. With pytest or API requests, verify one allowed call and one cross-institution denied call at central and source services. Verify unauthorized downloads fail and disabling the test account prevents subsequent protected access under the chosen revocation policy. Open the app and confirm job status changes.

Git and deployment checkpoint G1

Merge infrastructure first, then central and worker foundations in small PRs. CI runs formatting/linting, TypeScript checks, a frontend build, and focused backend checks without paid model calls. Tag v0.1-foundation. Optionally deploy a private development environment with synthetic records; local Compose remains the supported baseline.

Shared deployment responsibility: Member 2 owns environment wiring, Member 1 owns frontend configuration, and Member 3 owns worker/model settings. Never copy a member’s personal .env into the repository.

9 Stage 2 Source records identity and federation

Entry: authentication and services run. Exit: a referral can retrieve linked source records without crossing database boundaries. SRS focus: FR-06–10 and FR-16–21.

Source schemas and data import

Member 2 implements clinic patients/encounters/prescriptions/documents; lab patients/results/reports; specialist patients/incoming referrals/responses. Use the base dataset to populate coherent facts and dates. Add generated records only where needed. Build a reset-and-seed script that produces stable examples without silently wiping shared environments.

Patient linking and retrieval

Implement central PatientLink records with pending/verified/invalidated state, verification basis, authorized verifier, and history. Add lookup/link/review screens. On invalidation, prevent future use and identify affected drafts or approved packages for review; retain their historical evidence rather than silently rewriting them.

Source adapters expose the pinned FHIR subset. Suggested project endpoints: GET /patients/{id}/records and GET /patients/{id}/results. Label these custom endpoints returning FHIR content. Preserve units, clinical dates, record IDs and versions; keep original terms if unmapped. Central retrieval stores a source outcome for every request and deduplicates by source, record ID, and version.

Apply timeouts and bounded retries only to suitable failures. Denied access and invalid data need visible outcomes, not repeated blind retries. Member 1 connects patient selection and source-status screens; Member 3 consumes the shared evidence interface while building document ingestion.

Testing checkpoint T2

Use one linked patient, two different people with the same local ID, and one invalidated link. Confirm only verified links import evidence. Retrieve a result twice and confirm no duplicate. Stop the lab service and deny another request: both states must be distinct and existing clinic evidence must remain usable. Validate one representative resource of each implemented type with the HL7 validator.

Git and deployment checkpoint G2

Merge source services and fixtures, then adapters/importer and link UI. Run the shared identity/retrieval scenario before tagging v0.2-federation. Update the private development environment from that tag; record image/commit identifiers. If no host has been selected, demonstrate the same tagged release through Compose.

Handoff

Member 2 supplies actual authorized sample responses, error examples, and service setup notes. Members 1 and 3 switch off source stubs for the integrated path. Maintain a small stable fixture set for contracts even when the seed population grows.

10 Stage 3 Complete manual referral journey

Entry: authorized records are available. Exit: a clinician can manually prepare, approve, send, receive a response, and close one referral. This intermediate gate keeps the core workflow usable when AI fails.

Member 1 Workflow and approval

Implement draft CRUD/search, specialty, clinician-selected priority, planned procedure or explicit unknown state, evidence selection, clarification requests, and brief editing. Store separate clarification and delivery states. Validate legal state transitions on the server; a UI button is not the enforcement mechanism.

Create immutable ReferralPackage versions containing selected evidence, brief, structured flags when present, limitations, and approval metadata. Require expected version on edits to detect concurrent updates. Clinical edits after approval create a new draft/version and require renewed approval. Enforce required administrative fields and the agreed clinician override policy.

Member 2 Delivery and specialist service

Implement specialist receive, acknowledgment, accept/decline, consultation response draft, approval, and return endpoints. Use a delivery key based on sender/referral/package version; repeated requests return the original result. Approval is checked where the specialist response is owned, not merely in its screen.

Persist an outbox record in the same transaction as scheduling a send. A worker delivers it, records confirmation, and retries transient failures. A timed-out request may already have succeeded: use the same delivery key on retry. Incoming responses are also deduplicated.

Member 3 Manual evidence path

Provide manual transcription, source links, correction history, and clinical verification, without requiring successful OCR. Hand the shared evidence objects to Member 1. In parallel, continue OCR integration and rule evaluator preparation.

Testing checkpoint T3

Run one browser journey: create → approve → send → acknowledge → accept → respond → clinician review → follow-up completion → close. Try sending an unapproved package, editing an approved version, and saving a stale version; each must enforce its rule. Retry one send and return response and confirm no duplicates. Check the audit trail for the journey.

Git and deployment checkpoint G3

Merge vertical slices frequently. Tag the first passing end-to-end release v0.3-manual-flow and deploy a private staging instance if hosting is available. Run the same smoke journey after deployment. This is the first team demonstration checkpoint, not final SRS completion.

11 Stage 4 Documents OCR and evidence review

Entry: manual preparation works. Exit: supported documents can be extracted, corrected, and verified with source evidence. Owner: Member 3; Member 1 connects referral evidence controls. SRS focus: FR-22–34 and FR-58.

Processing pipeline

Validate upload and retain the original. Use embedded text where reliable for supported digital PDFs; rasterize scanned pages and perform the preprocessing needed by the chosen OCR. A line-recognition model requires a segmentation step. Run processing asynchronously and report queued/running/completed/failed states through GET /jobs/{id}.

Wrap the selected integration behind extract_document(). Store model/version, raw OCR text, page/region references, and available confidence metadata. Do not reinterpret an OCR confidence score as a calibrated clinical probability. Extraction proposes only the supported medication and laboratory fields; missing values remain missing.

Review interface and storage

Show the original page beside proposed fields. Users can correct, mark unresolved, exclude, or manually enter a field. Store EvidenceRevision records rather than overwriting raw extraction. Record author and time; clinical verification is a separate authorized action. Preserve original medication names, dose strings, lab values, units, event dates, and source references.

Use an explicit allowlist of evidence IDs and schema validation for language-model extraction. Treat document text as untrusted data; it cannot instruct the model to call tools, change permissions, or approve records. Invalid outputs and provider failures return visible review states and retain manual preparation.

Evaluation resources

Maintain a small development sample and a separate held-out sample with accurate transcriptions and expected fields. Label machine-generated layouts separately from actual handwriting. Record character/word error, supported field accuracy, and correction time; agree acceptance thresholds after the baseline. Never claim real handwriting reliability from clean synthetic PDFs.

Testing checkpoint T4

Use one supported scan, one digital PDF, one unreadable sample, and one rejected file. Verify extraction-to-correction-to-clinician-verification, original retention, and source navigation. Simulate model timeout; the user must still save manual evidence. Run the small held-out sample once after selecting the model, then only after material model/prompt changes.

Git and deployment checkpoint G4

Merge storage/review and OCR integration separately. Use fixture model responses in routine CI and a small real-model check for the stage gate. Tag v0.4-documents. Deploy the worker alongside the API, verify mounted storage and credentials, and smoke-test upload plus authenticated retrieval after deployment.

12 Stage 5 Medication mapping and deterministic flags

Entry: evidence/current-use verification and a clinically reviewed initial rule set are available. Exit: flags remain consistent, source-linked, versioned, and reviewable. SRS focus: FR-61–72.

Member 3 Mapping and rule evaluation

Implement a reviewed dictionary preserving the original name, ingredient/class identifiers, terminology source/version, and mapping status. RxNorm/RxClass may suggest mappings; verify the supported local brands and classes. Store current-use status separately from prescriptions: confirmed current, historical/stopped, uncertain, or conflicting. Record clinician verifier and time.

Implement evaluate_medications() as deterministic Python over typed input and versioned approved JSON rules. Each rule contains ID/version, class, specialty/procedure predicates, rationale/reference, reviewer, and approval/active state. Only approved mappings and rules can produce confirmed flags. Procedure-specific rules remain unevaluated if context is unknown.

Persist evaluation runs and flag versions. Show uncertain-use prompts and incomplete-history warnings alongside confirmed flags. Distinguish evaluation failure, unsupported coverage, and no matching rule. No-match wording must be limited to available verified records and supported rules; it is not treatment clearance.

Member 1 Review and approval integration

Display medication flags prominently in the draft and specialist view, independent of summary text. Provide Reviewed and Request clarification actions that record clinician, timestamp, flag version, and linked request. Acknowledgment neither approves treatment nor closes the referral. Freeze flags within the approved package.

Reevaluate when use status, mapping, specialty, procedure, or rule version changes. A material flag change creates a new review requirement and, for approved content, a new package version. Preserve earlier flags and acknowledgments. Member 2 transports the complete structured flag section without dropping versions or evidence references.

Testing checkpoint T5

Use a short table-driven pytest suite: confirmed matching use; stopped use; uncertain use; ambiguous mapping; missing procedure; denied source; no match; evaluation failure; and changed rule/context. Verify repeat evaluation creates no duplicate visible flags and a changed flag needs new acknowledgment. Manually inspect one displayed flag and its source. Clinical reviewer checks rule meaning, not only code execution.

Git and deployment checkpoint G5

Merge mapping/evaluator before approval/acknowledgment integration. Rule changes receive clinical review and code review; agents cannot self-approve clinical content. Tag v0.5-medication-flags after the test table passes. Deploy with the rule and mapping versions logged, then verify one confirmed and one uncertain case.

13 Stage 6 Brief generation completeness and follow through

Entry: reviewed evidence and structured flags exist. Exit: the full AI-assisted workflow works with clarification and follow-up. Owners: Member 3 for generation/checks, Member 1 for workflow/screens, Member 2 for source return communication.

Completeness and discrepancies

Implement the specialty checklist as versioned rules returning present, missing, unresolved, or clinician-marked not applicable. Member 3 produces findings; Member 1 stores assignment, request, response, clinical resolution, and rationale. Show both sources and dates for supported discrepancies. A changed value across time is not automatically a conflict. The checklist must not independently prescribe investigations.

Brief generation

Wrap the model behind generate_brief(). Send only the selected permitted evidence, source IDs, verification status, and explicit limitations. Require structured sections and evidence references for record-derived factual claims. Validate the output schema and reject references not in the supplied evidence set. Store prompt/model metadata and generated draft separately from edited and approved versions.

Render medication flags from the deterministic flag objects, never by asking the model to reproduce them faithfully. The model must not create, suppress, or change their meaning. If generation fails, show the error and keep the manual editor. An unavailable source must not become a claim of no medical history.

Return communication and closure

Finish specialist clarification, response approval/return, clinic receipt and separate clinician review, follow-up tasks with owners/status/optional due dates, and work queues for open or overdue work. Enforce closure only after response review and task disposition. Record reasons for cancellation, decline, and non-completion. Closure represents workflow completion.

Testing checkpoint T6

Run the complete journey with a missing item, one medication flag, a clarification response, generated brief, clinician edits, approval, specialist acknowledgment, and follow-up closure. Compare a few held-out briefs against an independently reviewed fact checklist for unsupported claims, omissions, source accuracy, and correction time. Do not use the same model as the sole judge of its own output.

Git and deployment checkpoint G6

Merge checklist/generation modules and connected screens incrementally. Tag v0.6-complete-flow when the integrated journey passes. Deploy to staging and repeat that one journey with the real source services and real selected model integrations. Capture a short demo and known limitations in the release notes.

14 Stage 7 Stabilization and demonstration release

Entry: the full workflow runs on staging. Exit: the tagged release passes the brief acceptance set and can be restored and demonstrated by another member.

Finish the operational details

Member 2 checks source outages, retries, credential configuration, access revocation, backups, and deployment scripts. Member 1 checks concurrency, version preservation, status transitions, keyboard use, clear error messages, and audit visibility. Member 3 checks model failures, rule failures, document retention, evidence access, and evaluation results.

Specify retention periods before hosted use. Apply expiry/deletion to permitted document copies, extraction data, and caches, with policy-defined handling for approved package evidence and audit history. Log only necessary metadata; avoid exposing clinical content or tokens in general application logs. Recheck permissions for queued jobs and protected downloads.

Testing checkpoint T7

- Run the existing focused automated checks plus three browser smoke cases: normal end-to-end referral; missing/denied source with manual fallback; changed medication flag requiring renewed review.
- Exercise one backup/restore into a disposable environment, including required file storage. Confirm one saved approved referral and its documents can be reopened.
- Run one lightweight 10-user screen/save test against the SRS's proposed 2-second p95 target. Exclude asynchronous OCR/LLM completion from ordinary request latency; record processing time separately.
- Confirm same local IDs remain distinct, unauthorized source and central requests are denied, approved content cannot be silently overwritten, and duplicate sends do not duplicate records. Reuse earlier tests rather than build a broad new suite.

Git and deployment checkpoint G7

Fix blockers through small PRs, rerun affected checks, and create v1.0-demo from passing main. Record the commit, images, schema migration revision, seed version, rule/mapping versions, and model configuration. Deploy that exact release to the selected private demo environment with TLS and external secrets.

Before upgrades, back up databases and files; apply reviewed migrations and readiness checks. After deployment, run login, patient retrieval, and one complete referral smoke journey. Retain the previous release images. Roll back application images only when schema-compatible; otherwise use the documented backup/restore procedure. Do not blindly reverse migrations or overwrite newer work.

Final handoff

Each member supplies module setup notes, endpoints, data ownership, checks, and limitations. The team delivers README quick start, .env.example, reset/seed instructions, API examples, supported FHIR fields, clinician-approved rule sources, model evaluation summary, deployment/restore commands, and a short demonstration script. Confirm another member can run the release without the author's laptop state.

15 Checkpoint summary and working resources

Stage	Minimum verification	Merge and deployment milestone
0 Scope	Sample JSON journey and model feasibility	Contracts merged; v0.0-contracts
1 Foundation	Fresh setup, allowed/denied access, job status	v0.1-foundation; optional private dev
2 Federation	Identity, adapter validation, outage, deduplication	v0.2-federation; update dev
3 Manual journey	One full referral plus approval/retry checks	v0.3-manual-flow; first staging demo
4 Documents	Upload, correction, failure, small held-out sample	v0.4-documents; deploy API and worker
5 Medication flags	Table-driven rule cases and changed review	v0.5-medication-flags; rule versions recorded
6 Full workflow	AI-assisted journey and brief fact checklist	v0.6-complete-flow; integrated staging demo
7 Demo release	Three smoke cases, restore, small load check	v1.0-demo; private release and smoke check

How much testing to do

On every PR: build/type/lint checks and the tests affected by that change. At each stage gate: its brief shared scenario and relevant earlier regression checks. On deployment: a short smoke check of the actual environment. Run paid model evaluation only at model/prompt changes or the AI stage gate. Expand checks only for a concrete failure or unverified SRS requirement.

Documentation and implementation resources

- Project requirements: updated FYP SRS (1).md, especially sections 4, 5, 8, 10, 11, and 12. Keep a versioned copy in docs/requirements and connect issues to FR identifiers.
- FastAPI documentation: https://fastapi.tiangolo.com/ — APIs, validation, OpenAPI. Keycloak: https://www.keycloak.org/securing-apps/oidc-layers — authentication integration.
- FHIR R4 resources: https://hl7.org/fhir/R4/resourcelist.html and validation: https://hl7.org/fhir/R4/validation.html — adapter contracts and resource checks.
- Synthea downloads: https://synthetichealth.github.io/downloads.html — linked synthetic base data. Preserve licenses and generated-data labels.
- RxNorm and RxClass: https://lhncbc.nlm.nih.gov/RxNav/APIs/index.html — terminology assistance; clinical rules still require independent review.
- OCR candidates: https://huggingface.co/microsoft/trocr-base-handwritten and https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/read — benchmark before selection.
- Celery tasks: https://docs.celeryq.dev/en/stable/userguide/tasks.html — job execution and retry behavior. Use the selected LLM provider's official structured-output documentation.

These resources support the proposed design; they do not replace clinical validation or imply production readiness. Keep the scope bounded to the chosen specialty, supported documents, and reviewed rules. Do not add a full hospital-management system, vector database, autonomous clinical agent, or comprehensive drug-interaction engine to this implementation plan.